SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

Aim: 1.A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques
1.B Design and implement algorithms to encrypt and decrypt messages using transposition techniques

1A classical technique (Caesar cipher)

```
def cipher_logic(text, shift, mode):
    """Core logic for encrypting and decrypting."""
    if mode == 'decrypt':
        shift = -shift

    result = ""
    for char in text:
        if char.isalpha():

            ascii_offset = 65 if char.isupper() else 97

            shifted_char = chr((ord(char) - ascii_offset + shift) % 26 + ascii_offset)
            result += shifted_char
        else:

            result += char

    return result

def main():
    print("--- Classical Substitution Cipher (CLI) ---")
    while True:
        mode = input("Do you want to (e)ncrypt or (d)ecrypt? (e/d/q to quit): ").lower()

        if mode == 'q':
            print("Exiting...")
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

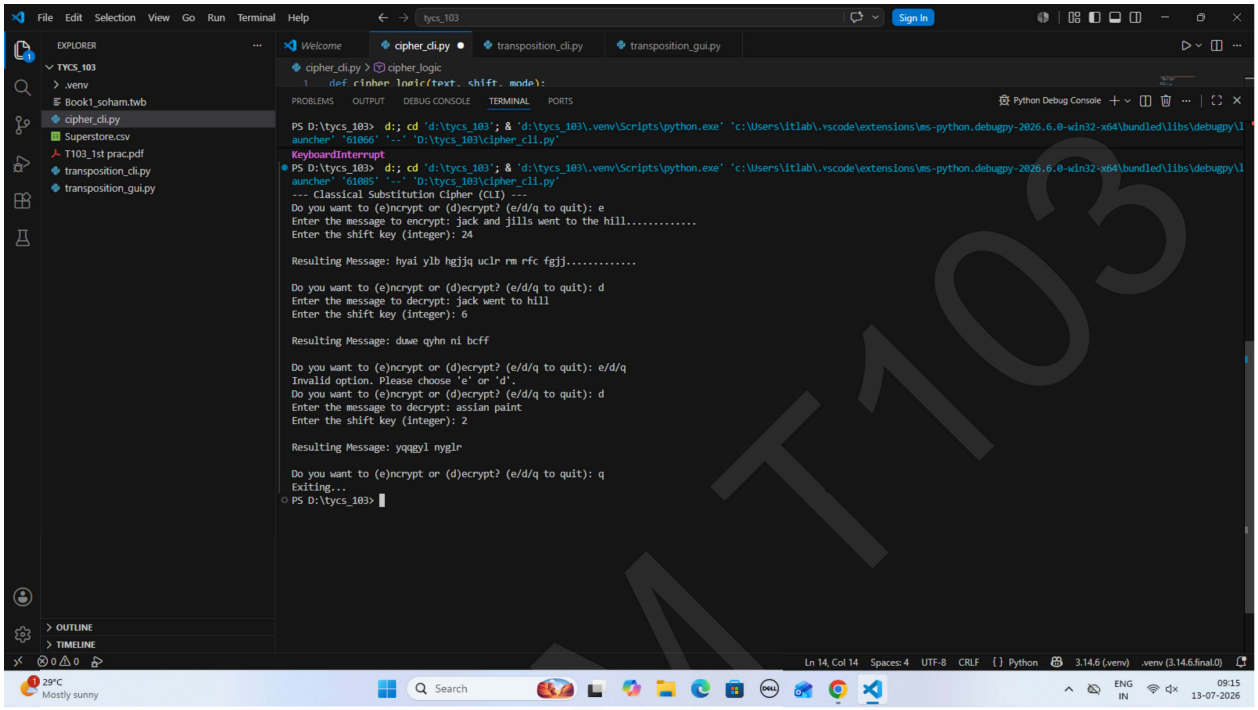
```
            break
        if mode not in ['e', 'd']:
            print("Invalid option. Please choose 'e' or 'd'.")
            continue

        operation = 'encrypt' if mode == 'e' else 'decrypt'

        message = input(f"Enter the message to {operation}: ")

        try:
            shift = int(input("Enter the shift key (integer): "))
        except ValueError:
            print("Shift key must be a valid integer.")
            continue

        result = cipher_logic(message, shift, operation)
        print(f"\nResulting Message: {result}\n")

if __name__ == "__main__":
    main()
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

OUTPUT

Encryption and Decryption



1.B transposition techniques (Rail Fence Cipher)

```
def rail_fence_cipher(text, key, mode):
    """Core logic for Rail Fence transposition."""
    if key < 2:
        return text
    if mode == 'encrypt':
        rails = [''] * key
        row = 0
        direction = 1


        for char in text:
            rails[row] += char
            row += direction
```

Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
        if row == 0 or row == key - 1:
            direction *= -1

    return "".join(rails)

elif mode == 'decrypt':
    # Step 1: Create a grid to map where the characters should go
    mark = [['\n' for _ in range(len(text))] for _ in range(key)]
    row = 0
    direction = 1

    for col in range(len(text)):
        mark[row][col] = '*'
        row += direction
        if row == 0 or row == key - 1:
            direction *= -1

    # Step 2: Fill the '*' markers with the actual cipher characters row by row
    idx = 0
    for r in range(key):
        for c in range(len(text)):
            if mark[r][c] == '*' and idx < len(text):
                mark[r][c] = text[idx]
                idx += 1

    # Step 3: Read the zigzag pattern to reconstruct the original message
    result = ""
    row = 0
    direction = 1
    for col in range(len(text)):
        result += mark[row][col]
        row += direction
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

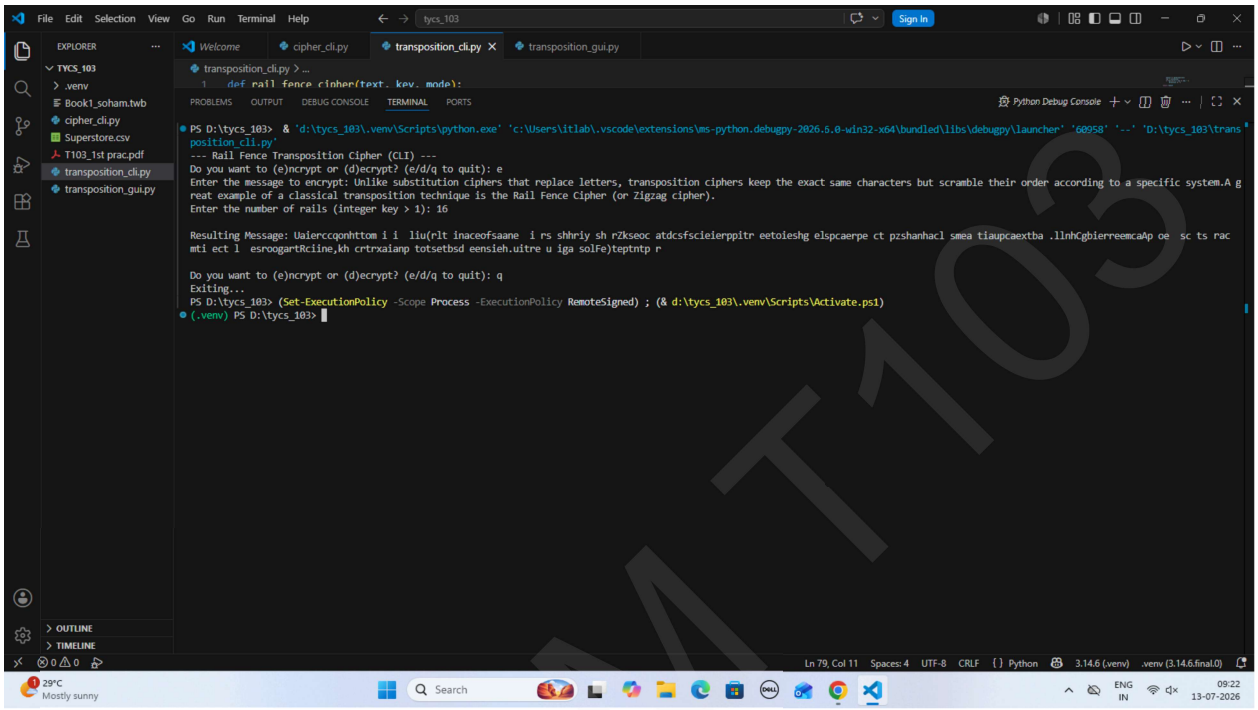
```
            if row == 0 or row == key - 1:
                direction *= -1

        return result

def main():
    print("--- Rail Fence Transposition Cipher (CLI) ---")
    while True:
        mode = input("Do you want to (e)ncrypt or (d)ecrypt? (e/d/q to quit): ").lower()

        if mode == 'q':
            print("Exiting...")
            break
        if mode not in ['e', 'd']:
            print("Invalid option. Please choose 'e' or 'd'.")
            continue

        operation = 'encrypt' if mode == 'e' else 'decrypt'
        message = input(f"Enter the message to {operation}: ")

        try:
            key = int(input("Enter the number of rails (integer key > 1): "))
        except ValueError:
            print("Key must be a valid integer.")
            continue

        result = rail_fence_cipher(message, key, operation)
        print(f"\nResulting Message: {result}\n")

if __name__ == "__main__":
    main()
```


Soham Patil
T103 Tycs

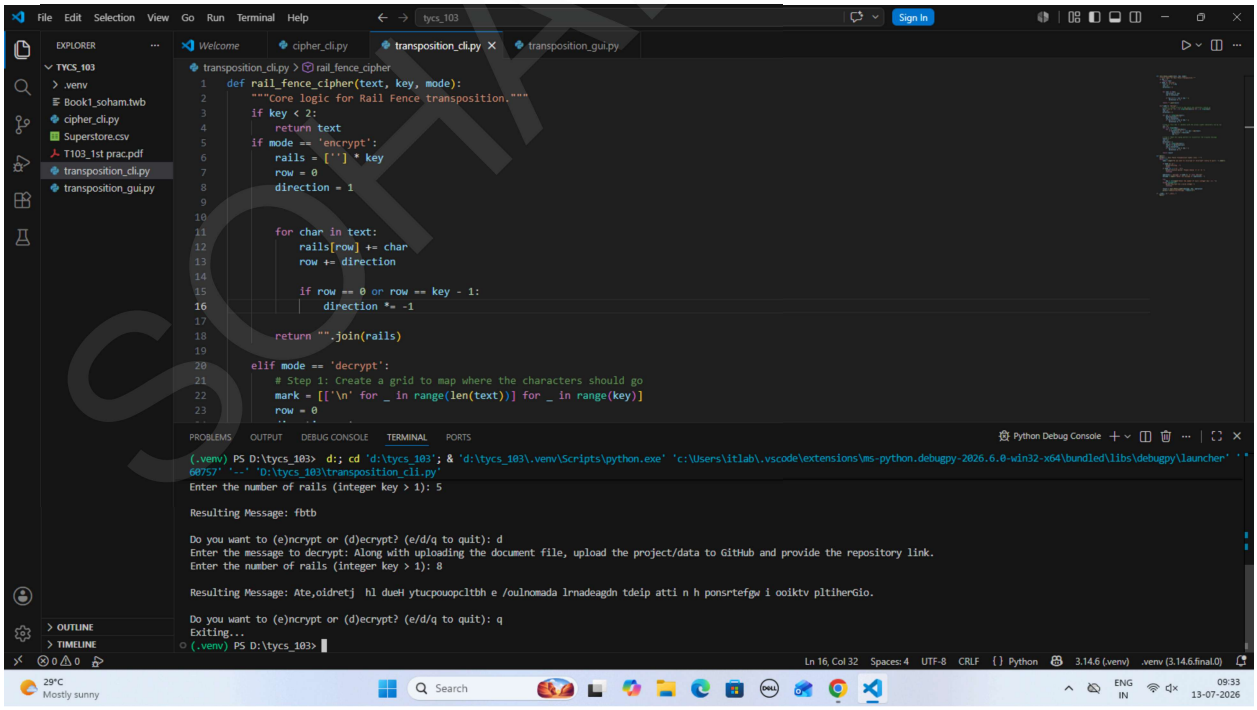
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

Output

Encryption



Decryption



Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

GUI FOR BOTH he Caesar (Classical Substitution) and Rail Fence (Transposition)

```
import customtkinter as ctk
from tkinter import messagebox

ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("blue")

# ==========================================
#         ALGORITHM IMPLEMENTATIONS
# ==========================================

def caesar_cipher(text, shift, mode):
    """Handles Caesar (Classical Substitution) Cipher."""
    if mode == 'decrypt':
        shift = -shift
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = 65 if char.isupper() else 97
            result += chr((ord(char) - ascii_offset + shift) % 26 + ascii_offset)
        else:
            result += char
    return result

def rail_fence_cipher(text, rails, mode):
    """Handles Rail Fence (Transposition) Cipher."""
    if rails < 2:
        return text

    if rails >= len(text):
        return text

    if mode == 'encrypt':
        fence = [[] for _ in range(rails)]
        rail, direction = 0, 1
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
        for char in text:
            fence[rail].append(char)
            rail += direction
            if rail == 0 or rail == rails - 1:
                direction *= -1
        return "".join(["".join(r) for r in fence])

    elif mode == 'decrypt':
        fence = [["\n"] * len(text) for _ in range(rails)]
        rail, direction = 0, 1
        for i in range(len(text)):
            fence[rail][i] = '*'
            rail += direction
            if rail == 0 or rail == rails - 1:
                direction *= -1

        idx = 0
        for r in range(rails):
            for c in range(len(text)):
                if fence[r][c] == '*' and idx < len(text):
                    fence[r][c] = text[idx]
                    idx += 1

        result = []
        rail, direction = 0, 1
        for i in range(len(text)):
            result.append(fence[rail][i])
            rail += direction
            if rail == 0 or rail == rails - 1:
                direction *= -1
        return "".join(result)


# ========================================
#           GUI APPLICATION
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
# ========================================

class CipherUI(ctk.CTk):
    def __init__(self):
        super().__init__()

        # --- Window Setup ---
        self.title("Cryptography Studio - Pro Edition")
        self.geometry("850x600")
        self.minsize(700, 500)

        # Configure Grid Layout
        self.grid_columnconfigure(0, weight=1)
        self.grid_rowconfigure(2, weight=1)

        self.build_header()
        self.build_settings_bar()
        self.build_text_areas()
        self.build_footer()

    def build_header(self):
        header_frame = ctk.CTkFrame(self, fg_color="transparent")
        header_frame.grid(row=0, column=0, padx=20, pady=(20, 10), sticky="ew")

        title = ctk.CTkLabel(header_frame, text="Cryptography Studio", font=ctk.CTkFont(size=28,
weight="bold"))
        title.pack(anchor="w")

        subtitle = ctk.CTkLabel(header_frame, text="Advanced text encryption & decryption tool",
text_color="gray")
        subtitle.pack(anchor="w")

    def build_settings_bar(self):
        settings_frame = ctk.CTkFrame(self, corner_radius=10)
        settings_frame.grid(row=1, column=0, padx=20, pady=10, sticky="ew")
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
        settings_frame.grid_columnconfigure(5, weight=1)

        # Cipher Selection (Only 2 options left now)
        ctk.CTkLabel(settings_frame, text="Algorithm:",
font=ctk.CTkFont(weight="bold")).grid(row=0, column=0, padx=(15, 5), pady=15)
        self.cipher_dropdown = ctk.CTkOptionMenu(settings_frame, values=["Caesar
(Substitution)", "Rail Fence (Transposition)"])
        self.cipher_dropdown.grid(row=0, column=1, padx=5, pady=15)

        # Key Input
        ctk.CTkLabel(settings_frame, text="Key:", font=ctk.CTkFont(weight="bold")).grid(row=0,
column=2, padx=(20, 5), pady=15)
        self.key_entry = ctk.CTkEntry(settings_frame, placeholder_text="e.g., 3", width=120)
        self.key_entry.insert(0, "3")
        self.key_entry.grid(row=0, column=3, padx=5, pady=15)



        self.btn_encrypt = ctk.CTkButton(settings_frame, text="🔒 Encrypt", fg_color="#2FA572",
hover_color="#1E7A52", command=lambda: self.execute_cipher('encrypt'))
        self.btn_encrypt.grid(row=0, column=5, padx=10, pady=15, sticky="e")

        self.btn_decrypt = ctk.CTkButton(settings_frame, text="🔓 Decrypt", fg_color="#C25A5A",
hover_color="#934141", command=lambda: self.execute_cipher('decrypt'))
        self.btn_decrypt.grid(row=0, column=6, padx=(0, 15), pady=15, sticky="e")

    def build_text_areas(self):
        text_frame = ctk.CTkFrame(self, fg_color="transparent")
        text_frame.grid(row=2, column=0, padx=20, pady=10, sticky="nsew")

        text_frame.grid_columnconfigure(0, weight=1)
        text_frame.grid_columnconfigure(1, weight=1)
        text_frame.grid_rowconfigure(1, weight=1)

        ctk.CTkLabel(text_frame, text="INPUT", font=ctk.CTkFont(size=14, weight="bold"),
text_color="gray").grid(row=0, column=0, sticky="w", padx=5)
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
        self.input_textbox = ctk.CTkTextbox(text_frame, font=ctk.CTkFont(family="Consolas",
size=14), corner_radius=10)
        self.input_textbox.grid(row=1, column=0, padx=(0, 10), pady=(5, 0), sticky="nsew")


        ctk.CTkLabel(text_frame, text="OUTPUT", font=ctk.CTkFont(size=14, weight="bold"),
text_color="gray").grid(row=0, column=1, sticky="w", padx=5)
        self.output_textbox = ctk.CTkTextbox(text_frame, font=ctk.CTkFont(family="Consolas",
size=14), corner_radius=10, fg_color="#2b2b2b")
        self.output_textbox.grid(row=1, column=1, padx=(10, 0), pady=(5, 0), sticky="nsew")

    def build_footer(self):
        footer_frame = ctk.CTkFrame(self, fg_color="transparent")
        footer_frame.grid(row=3, column=0, padx=20, pady=(10, 20), sticky="ew")
        footer_frame.grid_columnconfigure(0, weight=1)

        self.btn_clear = ctk.CTkButton(footer_frame, text="🗑 Clear All", fg_color="transparent",
border_width=1, text_color=("gray10", "gray90"), command=self.clear_all)
        self.btn_clear.grid(row=0, column=0, sticky="w")

        self.btn_copy = ctk.CTkButton(footer_frame, text="📋 Copy Output",
command=self.copy_to_clipboard)
        self.btn_copy.grid(row=0, column=1, sticky="e")

    def execute_cipher(self, mode):
        """Extracts values from UI, validates keys, processes data, and outputs result."""

        input_data = self.input_textbox.get("1.0", ctk.END).strip("\n")
        selected_algo = self.cipher_dropdown.get()
        key_raw = self.key_entry.get().strip()

        if not input_data:
            messagebox.showwarning("Empty Input", "Please type something into the Input Box
first.")
            return
```


Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
        try:
            key = int(key_raw)
        except ValueError:
            messagebox.showerror("Key Error", "The validation key must be a valid whole number
(integer).")
            return

        if "Caesar" in selected_algo:
            processed_text = caesar_cipher(input_data, key, mode)
        elif "Rail Fence" in selected_algo:
            if key < 2:
                messagebox.showerror("Key Error", "For Transposition Rail Fence, your rail key must
be 2 or higher.")
                return
            processed_text = rail_fence_cipher(input_data, key, mode)
        else:
            return

        # Write safely back into output panel view
        self.output_textbox.delete("1.0", ctk.END)
        self.output_textbox.insert(ctk.END, processed_text)

    def clear_all(self):
        self.input_textbox.delete("1.0", ctk.END)
        self.output_textbox.delete("1.0", ctk.END)
        self.key_entry.delete(0, ctk.END)
        self.key_entry.insert(0, "3")

    def copy_to_clipboard(self):
        output_data = self.output_textbox.get("1.0", ctk.END).strip("\n")
        if output_data:
            self.clipboard_clear()
            self.clipboard_append(output_data)
```

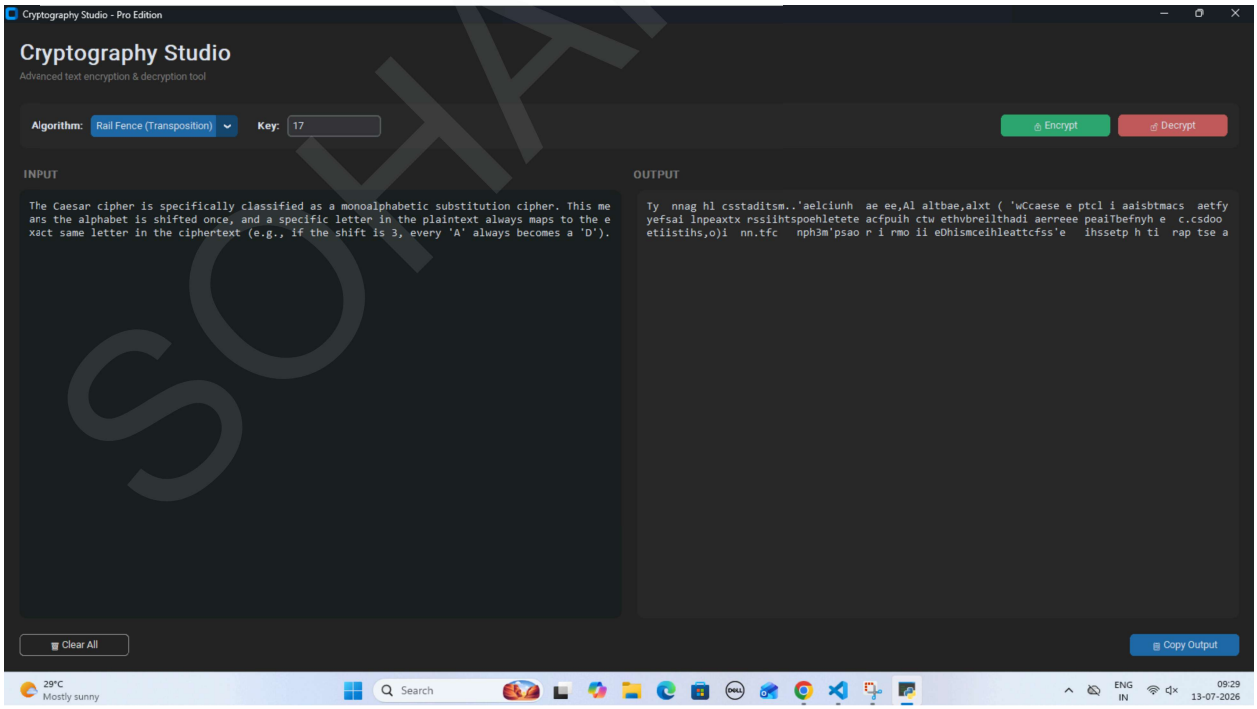

Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
        messagebox.showinfo("Success", "Output successfully copied to system clipboard!")
    else:
        messagebox.showwarning("Empty Data", "There is no text in the Output Box to copy.")

if __name__ == "__main__":
    app = CipherUI()
    app.mainloop()
```


OUTPUT

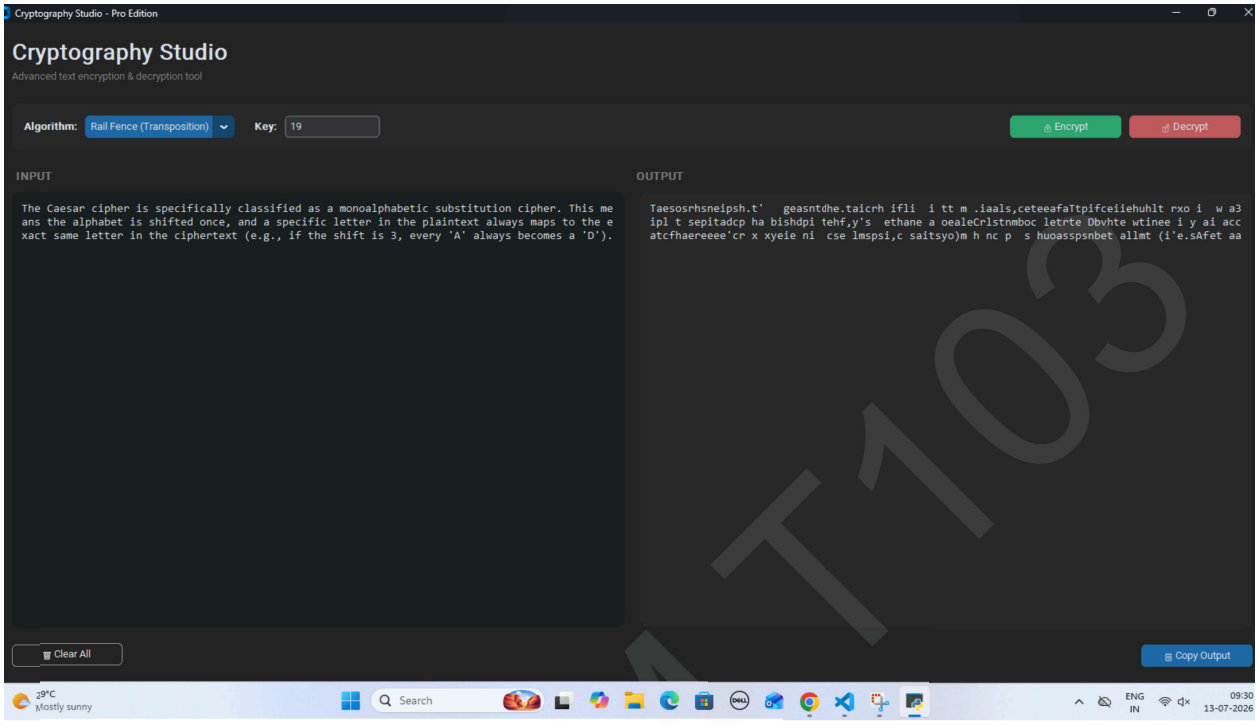
Transposition encryption



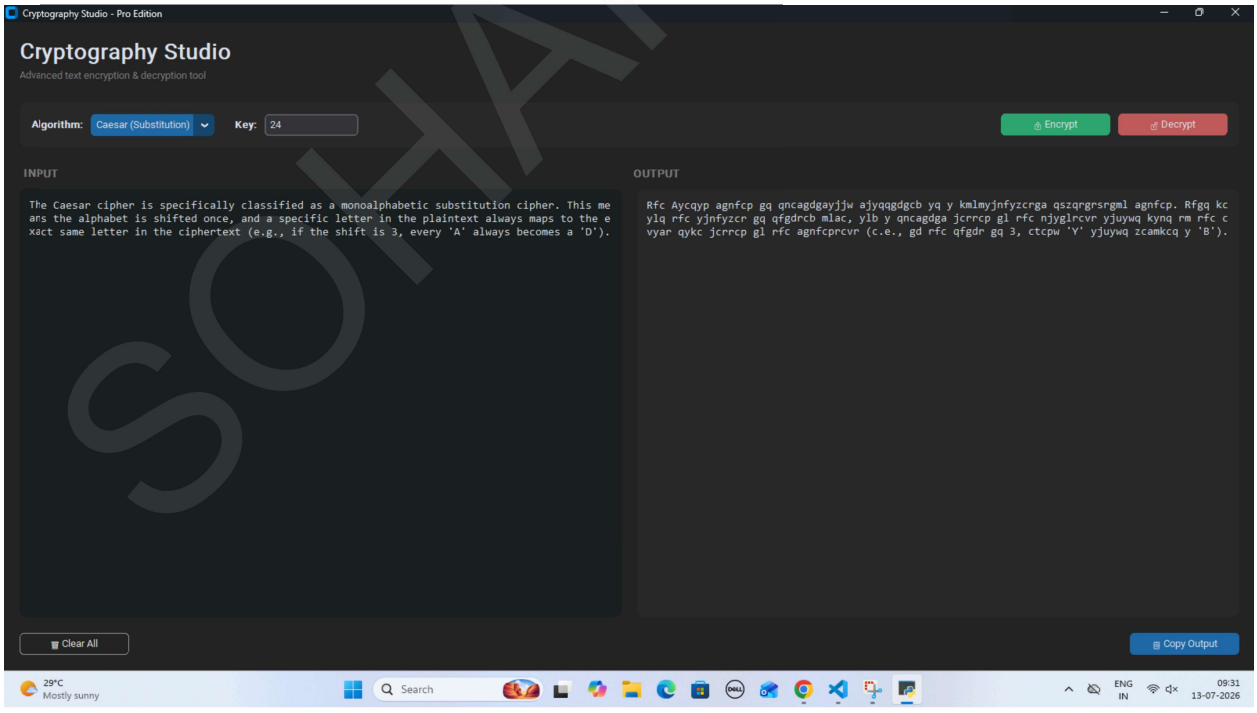
Transposition decryption

Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security



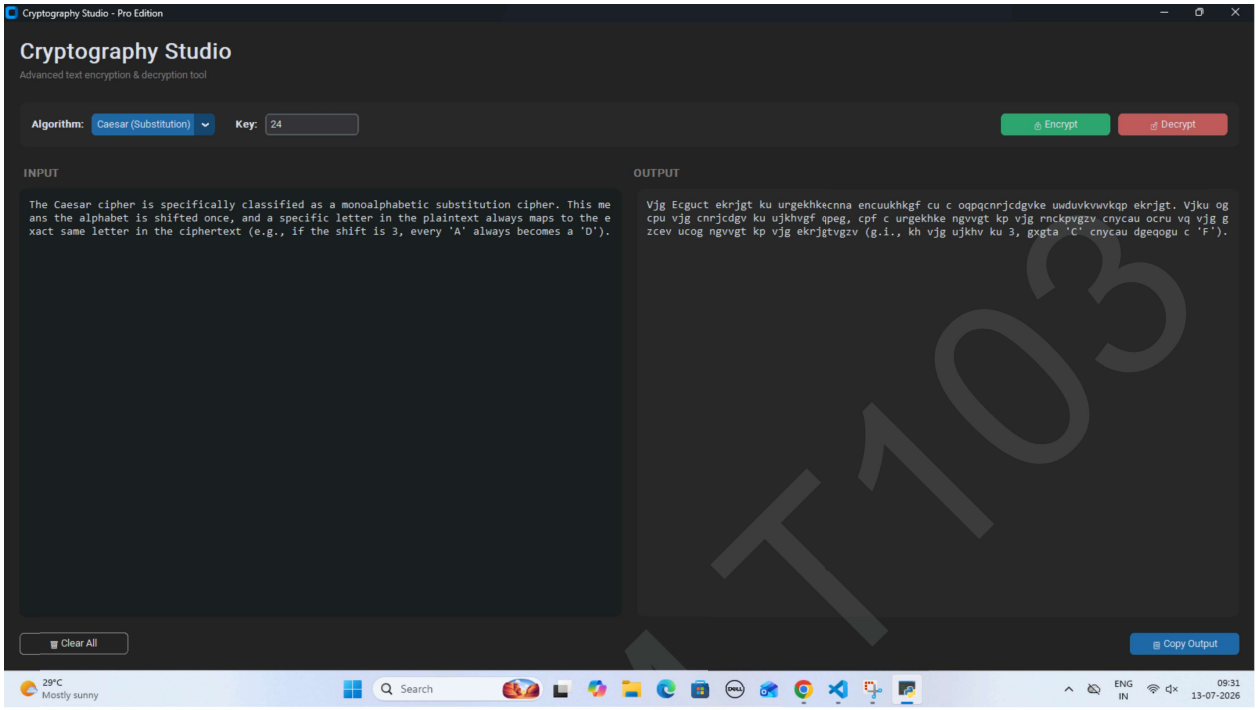
Caesar cipher encryption



Caesar cipher decryption

Soham Patil
T103 Tycs

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security



Soham Patil
T103 Tycs